Normalización y exploración de transacciones multifuente

Wilmer Salazar
KODIGO, Normalización y Estructura de Datos. Actividad individual
2 de noviembre de 2026
Repositorio: https://github.com/wkatir/genai-dev-critical-evaluation

1. Descripción general

El proyecto, *txnorm*, recibe registros de transacciones exportados por cuatro sistemas que describen un pago cada uno a su manera, los convierte a un modelo único, los valida y permite explorar el resultado desde una interfaz interactiva de terminal. Las cuatro fuentes son una API de banca central, una pasarela de pagos con tarjeta que reporta los montos en centavos y las fechas como marcas de tiempo Unix, un grupo de terminales POS de tienda con una exportación en formato local (`1.234,56`, `dd/mm/aaaa`, hora de México) y una billetera móvil que anida sus datos y usa códigos numéricos de estado.

Todo lo que depende de la forma de cada fuente está en un solo archivo de configuración, `config/rules.toml`: la ubicación de los campos, los formatos de número y de fecha, las zonas horarias, el mapeo de estados y la lista de monedas soportadas. Para agregar una quinta fuente basta con añadir una sección a ese archivo, sin tocar el código.

2. Ejecución

Se necesita Python 3.11 o superior. Desde una copia del repositorio:

```
pip install -r requirements.txt
pip install -e .
python -m txnorm                    # menú interactivo, carga todos los archivos de data/
python -m txnorm --summary          # métricas en JSON
python -m txnorm --export output    # JSON y CSV normalizados, rechazados y métricas
pytest                              # 40 pruebas
```

3. Modelo de datos

Cada registro válido se convierte en una *Transaction* con los campos de la tabla. Antes de eso, cada registro se procesa dentro de un contenedor más amplio que guarda también el dato original, la fuente detectada, la lista de problemas y una nota por campo que indica qué regla produjo su valor. Solo los registros sin errores pasan a ser una Transaction, de modo que el código que las usa nunca recibe valores faltantes o a medio procesar.

Campo	Tipo	Regla
id	texto	`fuente:source_id`; los identificadores solo son únicos dentro de su fuente
source_id	texto	identificador original, guardado como texto para conservar los ceros a la izquierda
amount	Decimal	unidades mayores, redondeado a los decimales de la moneda, siempre mayor que cero

Campo	Tipo	Regla
currency	texto	código ISO 4217 de la lista de monedas soportadas
status	enum	completed, pending, failed, cancelled o refunded
occurred_at	datetime	con zona horaria, almacenado en UTC
description, customer_id	texto	opcionales

4. Reglas de normalización

Detección de la fuente. Primero se busca un campo marcador explícito cuando la fuente lo tiene (por ejemplo `provider = "paygate"`). Si no lo hay, se identifica por estructura: gana la fuente cuyos campos obligatorios están todos presentes, la firma más específica resuelve los empates y, si el empate persiste, el registro se rechaza en lugar de adivinar.

Montos. Los montos se manejan como Decimal, nunca como punto flotante. Los montos en texto se leen con el separador decimal declarado para la fuente, así que `1.234,56` de un POS y `1,234.56` del banco se convierten en 1234.56. La agrupación de miles se verifica de forma estricta: un `1234.56` que llega de una fuente con coma decimal se rechaza, porque no se puede leer sin adivinar. Los centavos se dividen según la precisión de la moneda, y los valores con más decimales de los permitidos se redondean (redondeo bancario) y se marcan.

Moneda. Los códigos ISO se aceptan en mayúsculas o minúsculas, y los símbolos o nombres (`€`, `US$`, `euro`) pasan por una tabla de alias. Cada fuente puede redefinir un alias: `$` significa USD para el banco, pero MXN para las terminales de México.

Estado. Cada estado canónico enumera los valores originales que acepta ("succeeded", "Aprobada", "settled", etc.). La billetera agrega sus códigos numéricos a esa lista común. El cargador de la configuración rechaza un archivo en el que un mismo valor apunte a dos estados distintos.

Fechas. Cada fuente enumera los formatos que puede enviar y se prueban en orden: ISO 8601, segundos o milisegundos Unix, o patrones explícitos. Las horas sin desfase se interpretan en la zona horaria de la fuente (Ciudad de México, Bogotá); si la fuente no declara ninguna, se asume UTC y el registro se marca. Todas las fechas se guardan en UTC.

5. Validación y registros inválidos

Los registros inválidos no se eliminan. Pasan a una lista de cuarentena junto con todos los problemas encontrados y el dato original, y quedan fuera de todos los totales. La normalización no se detiene en el primer problema: un registro con una moneda inválida y sin fecha reporta ambas cosas. Dos situaciones son advertencias y no errores: un monto que hubo que redondear y una zona horaria que hubo que suponer. Esos registros siguen siendo válidos, pero aparecen marcados en la interfaz.

Problema	Código	Resultado

No es un objeto JSON; ninguna fuente coincide o coinciden varias	`MALFORMED_RECORD`
	`UNKNOWN_SOURCE`
	`AMBIGUOUS_SOURCE`
		rechazado

Campo obligatorio ausente o vacío; identificador de tipo incorrecto	`MISSING_FIELD` `INVALID_ID`	rechazado
Monto ilegible, cero o negativo	`INVALID_AMOUNT` `NON_POSITIVE_AMOUNT`	rechazado
Moneda o estado que no están en las reglas	`UNSUPPORTED_CURRENCY` `UNKNOWN_STATUS`	rechazado
Fecha ilegible, futura o anterior al año 2000	`INVALID_DATE` `DATE_OUT_OF_RANGE`	rechazado
Misma fuente e identificador ya aceptados	`DUPLICATE_ID`	rechazado
Monto redondeado; zona horaria supuesta	`AMOUNT_ROUNDED` `TIMEZONE_ASSUMED`	válido, marcado

Los montos negativos se rechazan porque la dirección del pago ya la indica el estado; permitir ambas cosas haría ambiguo un reembolso negativo. En cuanto a los duplicados, un identificador lo reclama el primer registro *válido* que lo usa, así que el reenvío corregido de un registro que antes fue rechazado sí se acepta.

6. Métricas e interfaz

El panel principal muestra cuántos registros se procesaron, cuántos son válidos y cuántos se rechazaron, el número de registros válidos por estado y los totales por moneda, tanto de todos los válidos como solo de los completados. Nunca se suman montos de monedas distintas, porque el proyecto no cuenta con una fuente de tipos de cambio.

La interfaz es una aplicación de terminal con menús, construida con la librería *rich*. Desde ella se pueden recorrer las transacciones por páginas, filtrar por estado, moneda, fuente, rango de fechas, rango de montos o texto libre, y ordenar los resultados; los totales bajo la tabla se ajustan a los filtros activos. La vista de detalle muestra cada valor normalizado junto a la regla que lo produjo y al registro original, que suele ser la forma más rápida de entender por qué un registro se aceptó o se rechazó. La vista de cuarentena lista los rechazados por código de problema, y los resultados se pueden exportar a JSON y CSV. El módulo de la interfaz solo muestra datos; toda la lógica está en los módulos del núcleo.

7. Uso de IA

Se usó Claude Code como compañero de programación. Propuso la estructura de módulos, escribió las primeras versiones de los analizadores, de la interfaz y de los datos de prueba, y ejecutó las pruebas después de cada cambio. Varias de sus sugerencias iniciales se cambiaron después de revisarlas:

- Los montos se leían primero como números de punto flotante. Se pasaron a Decimal, porque el punto flotante no puede representar con exactitud la mayoría de los valores en centavos.
- `$` se asignaba a USD en todas las fuentes. Se añadieron alias por fuente, ya que las terminales POS operan en México.
- El analizador de montos intentaba adivinar el separador decimal. Ahora usa el separador declarado para la fuente y rechaza cualquier valor ambiguo.
- Los reembolsos se modelaban como montos negativos. Ahora los montos son siempre positivos y el estado indica la dirección.
- La validación se detenía en el primer error. Ahora reúne todos los problemas del registro.
- El primer borrador de métricas sumaba todos los montos en un solo total. Ahora los totales se calculan por moneda.

Al probar a mano aparecieron dos errores en la interfaz: el aviso `[n]ext` desaparecía porque la librería de terminal lo interpretaba como marcado, y un solo valor inválido en los filtros cancelaba los filtros restantes. Ambos se corrigieron y quedaron cubiertos por pruebas.

8. Datos de prueba y verificación

`data/valid_transactions.json` contiene 20 registros correctos, cinco por fuente, que cubren todos los estados y monedas. `data/edge_cases.json` contiene 23 registros con problemas; cada uno lleva una nota con el resultado esperado, y una de las pruebas compara cada nota con lo que el programa hace realmente. Con ambos archivos, el programa procesa 43 registros: 24 son válidos (dos de ellos con advertencias) y 19 quedan en cuarentena. El conjunto tiene 40 pruebas que cubren los analizadores, las reglas de negocio, las métricas, los filtros, los errores de configuración y dos sesiones guionadas del menú interactivo.